PROJET PYTHON – MODULE TURTLE

Exercice 3 : L’arbre fractal

Analyse du code et réponses aux questions de réflexion

Nom	Muteba Wa Muteba
Prénom	Israël
Classe	L2 (CS)
Fichier analysé	`projet(exo3).py`
Date	12 novembre 2026

1. Présentation de l’exercice

Ce programme dessine un **arbre fractal** grâce à la **récursivité**. Le tronc mesure 100 pixels et se divise en deux branches plus courtes (70 % de la longueur précédente), inclinées de 60° à gauche et à droite. Chaque branche se divise à son tour, et ainsi de suite jusqu’à 7 niveaux. L’arbre obtenu compte $2^7 - 1 =$ **127 branches**.

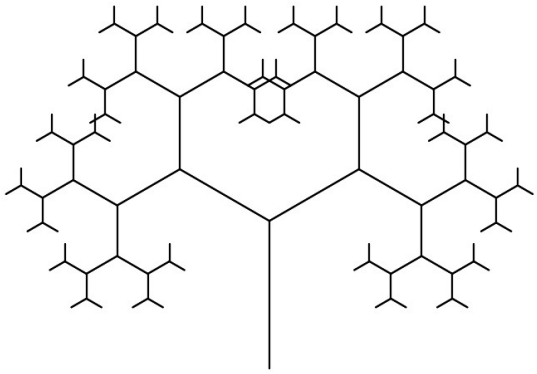


Figure 1 – *Résultat du programme avec arbre(100, 7) (illustration)*

2. Code source analysé

Fichier : projet(exo3).py

```
import turtle

t = turtle.Turtle()
t.speed(0)
t.left(90)
t.penup()
t.goto(0, -250)
t.pendown()

def arbre(longueur, niveau):
    if niveau == 0:
        return
    t.forward(longueur)
    t.left(60)
    arbre(longueur * 0.7, niveau - 1)
    t.right(120)
    arbre(longueur * 0.7, niveau - 1)
    t.left(60)
    t.backward(longueur)

arbre(100, 7)
turtle.done()
```

Fonctionnement pas à pas

Préparation : la tortue est tournée vers le haut (`t.left(90)`), puis placée en bas de la fenêtre (`t.goto(0, -250)`) sans laisser de trace grâce à `penup()` et `pendown()`.

Fonction récursive `arbre(longueur, niveau)` : une fonction récursive est une fonction qui s'appelle elle-même. Voici le rôle de chaque instruction :

Étape	Instruction	Rôle
1	`if niveau == 0: return`	**Cas de base** : il n'y a plus rien à dessiner, la fonction s'arrête.
2	`t.forward(longueur)`	Trace la branche courante.
3	`t.left(60)`	Tourne vers la gauche pour préparer la branche gauche.
4	`arbre(longueur * 0.7, niveau - 1)`	**Appel récursif** : dessine tout le sous-arbre gauche, avec une branche plus courte (70 %) et un niveau de moins.
5	`t.right(120)`	Pivote de 120° vers la droite : la tortue passe de +60° à -60° par rapport à la branche.
6	`arbre(longueur * 0.7, niveau - 1)`	Dessine le sous-arbre droit.
7	`t.left(60)`	Remet la tortue dans la direction d'origine de la branche.
8	`t.backward(longueur)`	Recule jusqu'à la base de la branche pour que l'appel précédent puisse continuer.

Lancement : `arbre(100, 7)` dessine l'arbre complet, puis `turtle.done()` garde la fenêtre ouverte.

3. Réponses aux questions d'analyse

Question 1 – Quelles instructions sont répétées ? Pourrait-on utiliser une boucle ?

La même séquence d'instructions se répète à **chaque niveau** : avancer, tourner, dessiner un sous-arbre, pivoter, dessiner un deuxième sous-arbre, revenir. Cette répétition est de nature **auto-similaire** : chaque branche est elle-même un petit arbre. C'est pourquoi le programme utilise la **récursivité** plutôt qu'une simple boucle : le nombre de branches double à chaque niveau (1, 2, 4, 8, ...) et une boucle `for` ordinaire décrit mal cette multiplication.

Dans le code, on remarque deux répétitions précises.

- **L'appel** `arbre(longueur * 0.7, niveau - 1)` **est écrit deux fois**, une fois pour la branche gauche et une fois pour la branche droite.
- **L'instruction** `t.left(60)` **apparaît deux fois** : la première tourne vers la gauche pour aller dessiner, la seconde remet la tortue dans sa direction d'origine.

Ces deux appels peuvent justement être regroupés dans une petite boucle sur le sens de rotation :

Variante avec une boucle (même dessin)

```
def arbre(longueur, niveau):
    if niveau == 0:
        return
    t.forward(longueur)
    for sens in (1, -1):          # 1 = gauche, -1 = droite
        t.left(60 * sens)
        arbre(longueur * 0.7, niveau - 1)
        t.right(60 * sens)
    t.backward(longueur)
```

Pour chaque valeur de `sens`, la tortue tourne, dessine un sous-arbre, puis revient dans la direction d'origine. Le résultat est identique, avec une instruction de moins.

> **À retenir** : ici la « boucle » est assurée par la récursivité, avec la variable `niveau` qui joue le rôle d'un compteur qui diminue jusqu'à 0. Sans récursivité, il faudrait imbriquer autant de boucles que de niveaux, ce qui est impossible à écrire pour un niveau choisi librement.

Question 2 – Une partie du dessin pourrait-elle être isolée dans une fonction ?

Oui, et c'est déjà le cas : la fonction `arbre()` isole le motif « une branche et ses deux sous-arbres ». Le reste du programme se limite à placer la tortue et à lancer le dessin avec `arbre(100, 7)`. La fonction est indispensable ici, car la récursivité n'existe que grâce aux fonctions.

On peut aller plus loin en isolant le **placement de la tortue** et en passant l'**angle** et le **facteur de réduction** en paramètres :

Proposition : fonction plus générale

```
def aller_au_depart(x, y):
    t.penup()
    t.goto(x, y)
    t.pendown()

def arbre(longueur, niveau, angle=60, reduction=0.7):
    if niveau == 0:
        return
    t.forward(longueur)
    t.left(angle)
    arbre(longueur * reduction, niveau - 1, angle, reduction)
    t.right(2 * angle)
    arbre(longueur * reduction, niveau - 1, angle, reduction)
    t.left(angle)
    t.backward(longueur)

t.left(90)
aller_au_depart(0, -250)
arbre(100, 7, angle=40)
```

Les valeurs `60` et `0.7` ne sont plus « cachées » dans le code : elles deviennent des réglages que l'on change en une seule ligne, sans toucher au corps de la fonction.

Question 3 – Quels paramètres pouvez-vous modifier pour changer le résultat ?

Paramètre	Où le modifier	Effet sur le dessin
`longueur` (100)	1er argument de `arbre()`	Longueur du tronc, donc taille totale de l'arbre.
`niveau` (7)	2e argument de `arbre()`	Nombre de niveaux de branches : plus il est grand, plus l'arbre est fourni.
`0.7`	Dans les appels récursifs	Facteur de réduction : 0,5 donne un arbre maigre, 0,8 un arbre dense et plus haut.
`60`, `120`, `60`	Rotations dans `arbre()`	Écartement des branches. Règle à respecter : `right` = 2 × `left`.
`t.goto(0, -250)` et `t.left(90)`	Début du programme	Position de départ et orientation du tronc.
`t.speed()`, `pensize()`, `color()`	Début du programme	Vitesse, épaisseur et couleur du trait (noir par défaut).

Le nombre de branches vaut 2 puissance `niveau`, moins 1 (en Python : `2 ** niveau - 1`). Il augmente donc très vite :

`niveau`	1	3	5	7	9	10
Nombre de branches	1	7	31	127	511	1 023

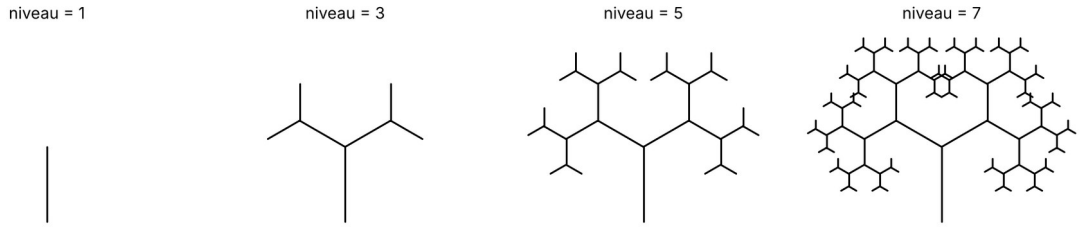


Figure 2 – *Évolution de l'arbre selon le niveau : 1, 3, 5 et 7 (même échelle)*

L'**angle** change fortement l'allure de l'arbre : un petit angle donne un arbre élancé, tandis qu'un angle de 60° produit des branches très ouvertes qui se croisent presque.

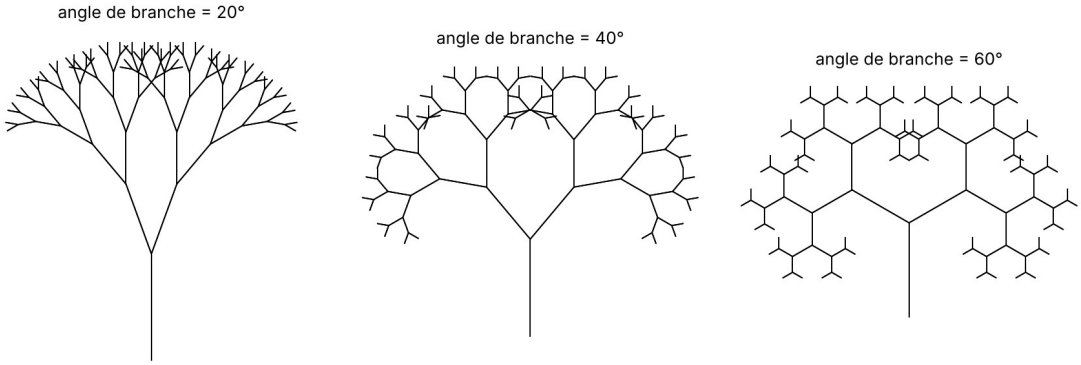


Figure 3 – *Effet de l'angle de branche : 20°, 40° et 60° (niveau 7)*

Pour obtenir un tronc plus épais qui s'amincit vers le haut, on peut aussi relier l'épaisseur du trait au niveau :

Amélioration possible

```
def arbre(longueur, niveau):
    if niveau == 0:
        return
    t.pensize(niveau)        # tronc épais, petites branches fines
    t.forward(longueur)
    # ... le reste de la fonction ne change pas
```

Question 4 – Quelles nouvelles instructions ou notions avez-vous dû découvrir ?

Notion	Explication	Exemple dans le code
Récursivité	Une fonction qui s'appelle elle-même, sur un problème plus petit	`arbre(longueur * 0.7, niveau - 1)`
Cas de base	Condition d'arrêt obligatoire : sans elle, la fonction s'appellerait sans fin	`if niveau == 0:`
`return`	Quitte la fonction immédiatement	`return`
Retour au point de départ	Chaque appel doit ramener la tortue à sa position initiale pour que l'appel précédent continue correctement	`t.backward(longueur)`
Fonction à plusieurs paramètres	Les valeurs passées changent à chaque appel (branche plus courte, niveau plus petit)	`arbre(longueur, niveau)`
Nouvelles méthodes de turtle	`penup()`, `pendown()`, `goto(x, y)`, `backward()`, `right()`	`t.goto(0, -250)`

Notion	Explication	Exemple dans le code
`==` et `=`	`==` compare deux valeurs, `=` range une valeur dans une variable	`niveau == 0`
Fractale	Figure dont chaque partie ressemble à l'ensemble	L'arbre complet

Pour bien comprendre la récursivité, voici un exemple très simple : un compte à rebours.

Exemple : compte à rebours récursif

```
def compte_a_rebours(n):
    if n == 0:              # cas de base
        print("Fini !")
        return
    print(n)
    compte_a_rebours(n - 1)   # appel récursif avec un nombre plus petit

compte_a_rebours(3)         # affiche 3, 2, 1, puis Fini !
```

Dans l'arbre, c'est le même mécanisme : `niveau` diminue de 1 à chaque appel jusqu'à atteindre 0, ce qui arrête la récursivité. Si l'on oublie le cas de base, Python s'arrête avec l'erreur `RecursionError: maximum recursion depth exceeded`.

Question 5 – Qu'avez-vous essayé lorsque le résultat n'était pas celui attendu ?

Problème observé	Hypothèse ou essai	Solution retenue
`IndentationError: expected an indented block after function definition`	Le corps de `arbre()` n'était pas décalé vers la droite	Indentation de 4 espaces de toutes les lignes de la fonction
Erreur de syntaxe sur la ligne `for niveau == 0:`	`for` sert à répéter, pas à tester une condition	Remplacement par `if niveau == 0:`
Premières versions avec des branches à 30° (`left(30)`, `right(60)`, `left(30)`)	Comparer avec d'autres angles pour voir l'effet sur la forme	Valeurs finales 60 / 120 / 60
Dessin lent ou trop chargé avec un niveau élevé	Le nombre de branches double à chaque niveau	`t.speed(0)` et test avec de petits niveaux (1, 3, 5) avant d'atteindre 7
Fichier de travail nommé `is.py`	`is` est un mot réservé de Python	Choisir un nom sans mot réservé

Voici les deux erreurs de la première version et leur correction :

Avant (erreurs)

```
def arbre(longueur, niveau):
for niveau == 0:
return
t.forward(longueur)
```

Après (corrigé)

```
def arbre(longueur, niveau):
    if niveau == 0:
        return
    t.forward(longueur)
```

Pour **comprendre le rôle de chaque instruction**, une expérience utile consiste à supprimer temporairement `t.backward(longueur)` et à observer le résultat : la tortue ne revient plus à la base de la branche, et les sous-arbres suivants sont dessinés au mauvais endroit.

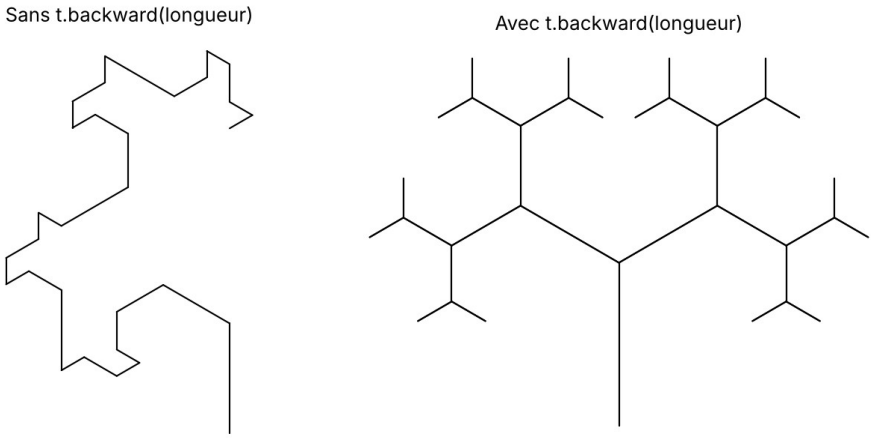


Figure 4 – *À gauche, sans t.backward(longueur) : le dessin est désorganisé. À droite, avec t.backward(longueur) : l'arbre est correct (niveau 5)*

Méthode suivie : lire le message d'erreur et le numéro de ligne, tester avec un **petit niveau** (1, 2, 3) pour vérifier le dessin à la main, puis augmenter progressivement, et enfin supprimer ou modifier une instruction à la fois pour comprendre son effet.

4. Conclusion

Cet exercice m'a fait découvrir la **récursivité** : une fonction qui se rappelle elle-même sur un problème plus petit, avec un **cas de base** qui arrête le processus. Avec seulement une dizaine de lignes, on obtient un arbre de 127 branches. J'ai aussi compris l'importance de `t.backward(longueur)`, qui ramène la tortue à sa place après chaque branche, et le rôle des paramètres (longueur, niveau, angle, facteur de réduction) pour varier l'aspect de l'arbre.

Document rédigé par **Muteba Wa Muteba Israël** – Classe L2 (CS).